

Tarea 2: Hemograma Automatizado

Informe Final

Nombre Integrante 1

Nombre Integrante 2

Abril 2026

Índice

1. Introducción	3
2. Resumen del sistema propuesto	3
2.1. Descripción general del pipeline	3
2.2. Decisiones de diseño principales	4
3. Dashboards públicos de Weights & Biases	4
3.1. Enlaces	5
4. Entrenamiento del detector	5
4.1. Configuración del detector	5
4.2. Curvas de entrenamiento del detector	6
4.3. Evolución del mAP	6
4.4. Resultados finales del detector	7
4.5. Curvas de pérdida de entrenamiento y validación	7
4.6. Análisis de entrenamiento	8
5. Evaluación comparativa de clasificadores	8
5.1. Metodología de comparación	8
5.2. Matriz de confusión de la ANN	10
5.3. Matriz de confusión de la SVM	10
5.4. Métricas por clase	11
5.5. Comparación global	11
5.6. Análisis comparativo	11
6. Justificación empírica de la mitigación de la alta dimensionalidad	12
6.1. Problema	12
6.2. Mecanismo evaluado	12

6.3. Evaluación mediante sweeps	13
6.4. Decisión final	13
7. Barrido de hiperparámetros con W&B (FR-11)	13
7.1. Metodología del barrido	13
7.2. Resumen de sweeps ejecutados	14
8. Resultados asociados a FR-14	14
8.1. Versión original del requisito	14
8.2. Versión actualmente aplicada en el proyecto	14
8.3. Metodología de evaluación usada	14
8.4. Resultados	15
8.5. Discusión	15
8.6. Metodología de medición	16
8.7. Hardware utilizado	16
8.8. Tiempos por etapa	16

1. Introducción

Este documento presenta el desarrollo del pipeline de hemograma automatizado propuesto para la Tarea 2 del curso EL 5857 Aprendizaje Automático. El sistema integra detección de objetos, extracción de características, clasificación multiclase, análisis comparativo de clasificadores y evaluación integral del pipeline.

El objetivo general del proyecto es procesar imágenes microscópicas del conjunto de datos BCCD para detectar células sanguíneas, clasificarlas en las categorías *Platelets*, *RBC* y *WBC*, y posteriormente analizar la distribución celular obtenida mediante un contraste estadístico respecto a un baseline derivado del conjunto de entrenamiento. Para ello se diseñó una arquitectura modular que permite ejecutar el sistema por etapas, desde el entrenamiento del detector y los clasificadores hasta la inferencia end-to-end sobre imágenes reales.

Durante el desarrollo, el alcance del proyecto cambió con respecto a la idea inicial. El curso dejó de contemplar el uso de un conjunto sintético provisto externamente, por lo que la validación pasó a centrarse completamente en datos reales del dataset BCCD. Este cambio hizo que la evaluación final se enfocara en comparar tres niveles de etiquetado sobre imágenes reales: la etiqueta real anotada en el dataset, la etiqueta predicha por YOLO y la etiqueta emitida por el clasificador posterior sobre cada recorte detectado.

2. Resumen del sistema propuesto

2.1 Descripción general del pipeline

El sistema propuesto sigue un flujo secuencial y modular. En una primera etapa, un detector YOLO entrenado sobre BCCD recibe una imagen microscópica completa y produce cajas delimitadoras junto con una clase preliminar para cada célula detectada. A partir de estas detecciones, se recortan regiones de interés correspondientes a células individuales, las cuales son transformadas a un tamaño uniforme y procesadas mediante un extractor de características basado en ResNet18 truncada.

En una segunda etapa, los vectores de características extraídos se normalizan y se utilizan como entrada para clasificadores multiclase entrenados de forma independiente. En este proyecto se compararon una red neuronal artificial (ANN) y una máquina de soporte vectorial (SVM), con el fin de seleccionar la configuración con mejor desempeño. Finalmente, sobre la salida del clasificador se calculan conteos y proporciones celulares por imagen, y estas se contrastan contra un baseline estadístico construido a partir del conjunto de entrenamiento. Esto permite no solo obtener predicciones por célula, sino también una caracterización agregada de cada imagen y una comparación entre etiqueta real, etiqueta YOLO y etiqueta del clasificador.

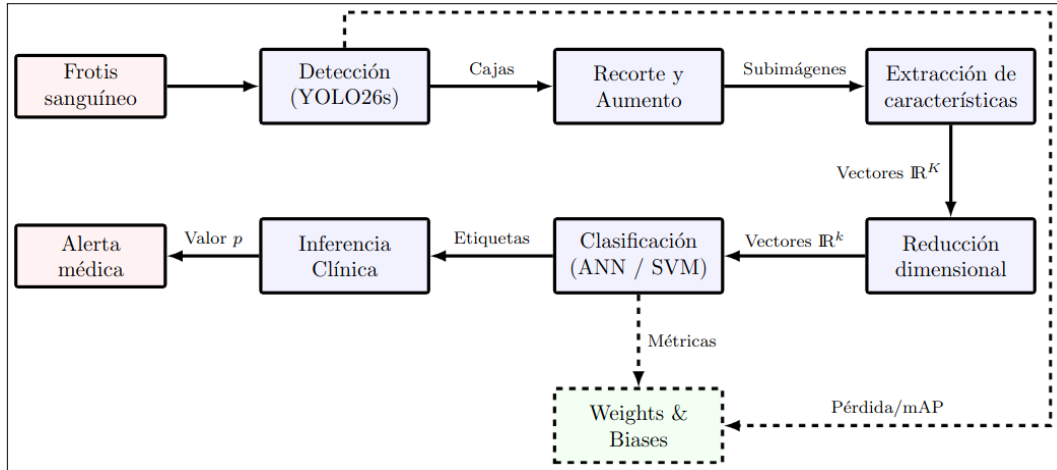


Figura 1: Flujo de datos y arquitectura de módulos del sistema de diagnóstico automatizado

2.2 Decisiones de diseño principales

- **Detector de objetos utilizado:** se utilizó YOLO26s como detector principal, ajustado sobre el dataset BCCD para la localización de células sanguíneas en imágenes completas.
- **Extractor de características empleado:** se utilizó ResNet18 preentrenada, truncada en `layer3`, como extractor de características profundas para los recortes celulares.
- **Clasificadores comparados:** se entrenaron y compararon dos enfoques de clasificación multiclase: una red neuronal artificial (ANN) y una máquina de soporte vectorial (SVM).
- **Estrategia usada para mitigar la alta dimensionalidad:** se implementó una proyección lineal opcional como mecanismo de reducción dimensional. Sin embargo, bajo el protocolo experimental utilizado, esta estrategia no produjo mejora empírica respecto a la configuración sin proyección, por lo que no se adoptó como opción final.
- **Herramientas de monitoreo y registro experimental:** se utilizó Weights & Biases (W&B) para registrar corridas, configuraciones, métricas y artifacts asociados al detector y a los clasificadores.

Como decisión de diseño general, se priorizó una arquitectura modular y ejecutable por etapas. Esto permitió entrenar, evaluar y depurar de forma independiente el detector, el extractor, los clasificadores y el pipeline de inferencia. También se mantuvo un archivo de configuración central en YAML para asegurar reproducibilidad y consistencia experimental entre corridas.

3. Dashboards públicos de Weights & Biases

Esta sección reúne los enlaces directos a los tableros públicos de W&B asociados al entrenamiento del detector y a la exploración de clasificadores.

3.1 Enlaces

- Dashboard detector YOLO: <https://wandb.ai/t2-ak-tec/bccd-t2-ak>
- Dashboard ANN: <https://wandb.ai/t2-ak-tec/bccd-t2-ak>
- Dashboard SVM / experimentos comparativos: <https://wandb.ai/t2-ak-tec/bccd-t2-ak>
- Sweep o registro equivalente de exploración: <https://wandb.ai/t2-ak-tec/bccd-t2-ak>

4. Entrenamiento del detector

4.1 Configuración del detector

El detector de objetos utilizado en este proyecto fue **YOLO26s**, una variante compacta de la familia YOLO adecuada para entrenamiento y evaluación en un entorno con recursos de cómputo limitados. Como punto de partida se emplearon pesos preentrenados en COCO, específicamente el archivo `yolo26s.pt`, con el fin de aprovechar transferencia de aprendizaje y reducir el costo de entrenamiento desde cero.

La configuración principal de entrenamiento del detector se centralizó en el archivo `configs/config.yaml`. En la configuración base utilizada para las corridas principales se definieron los siguientes hiperparámetros:

- **Modelo usado:** YOLO26s.
- **Pesos iniciales:** `yolo26s.pt`.
- **Número de épocas:** 15.
- **Tamaño de imagen:** 320.
- **Batch size:** 4.
- **Tasa de aprendizaje inicial:** 0.01.

Durante la inferencia del detector se utilizaron además parámetros de operación específicos, en particular un umbral de confianza y un umbral de IoU para el postprocesamiento de detecciones. Estos parámetros se estudiaron posteriormente en la etapa de evaluación comparativa, ya que afectan de forma importante el balance entre falsos positivos y falsos negativos.

El criterio para seleccionar el mejor modelo fue el checkpoint `best.pt` generado automáticamente por Ultralytics durante el entrenamiento. Este archivo corresponde al modelo con mejor desempeño sobre validación según la métrica interna de selección del framework, la cual prioriza el rendimiento del detector en el conjunto de validación a lo largo de las épocas.

4.2 Curvas de entrenamiento del detector

La Figura 2 muestra la evolución de las pérdidas principales del detector durante el entrenamiento. En particular, se monitorearon las pérdidas de caja delimitadora (**box loss**), clasificación (**cls loss**) y distribución focal (**dfl loss**), tanto en entrenamiento como en validación. Estas curvas permiten verificar que el modelo efectivamente aprende una representación útil del problema y que el entrenamiento se mantiene estable.

En las corridas realizadas se observó una reducción progresiva de las pérdidas de validación, especialmente en **val/cls_loss**, lo que indica que el detector logró ajustar razonablemente bien la separación entre clases. También se observó que la clase *Platelets* resultó más difícil que *RBC* y *WBC*, lo cual es consistente con el desbalance del dataset y con la dificultad visual de esta categoría.

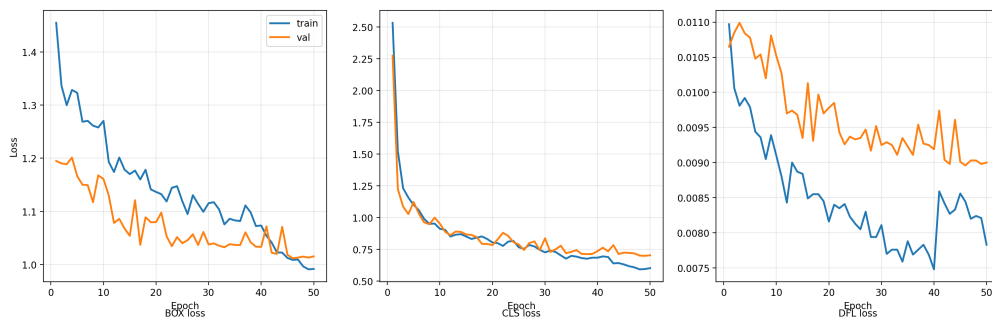


Figura 2: Curvas de entrenamiento del detector

4.3 Evolución del mAP

Para evaluar el desempeño del detector se monitorearon métricas estándar de detección, particularmente **precision**, **recall**, **mAP50** y **mAP50-95** sobre el conjunto de validación. La Figura 3 resume la evolución de estas métricas a lo largo del entrenamiento.

Las curvas muestran un comportamiento relativamente estable y consistente con una convergencia útil del detector. En particular, el **mAP50** aumentó progresivamente hasta estabilizarse alrededor de 0.91–0.92, mientras que el **mAP50-95** se mantuvo en el rango de 0.62–0.64 en las últimas épocas. Asimismo, la precisión y el recall evolucionaron de forma razonablemente balanceada, lo que indica que el detector no solo aprendió a localizar objetos, sino también a mantener un compromiso aceptable entre detecciones correctas y omisiones.

En la corrida final disponible del detector se obtuvo un valor de **mAP50** = 0.91923 y **mAP50-95** = 0.63881, acompañado de **precision** = 0.85414 y **recall** = 0.87606. Estos resultados indican que el modelo fue capaz de localizar y clasificar células con buena calidad general, aunque con diferencias entre clases. En particular, *WBC* mostró el comportamiento más fuerte, mientras que *Platelets* continuó siendo la clase más desafiante. Esta diferencia fue relevante en etapas posteriores del proyecto, ya que parte de los errores del sistema end-to-end se explican más por limitaciones de detección que por el clasificador posterior.

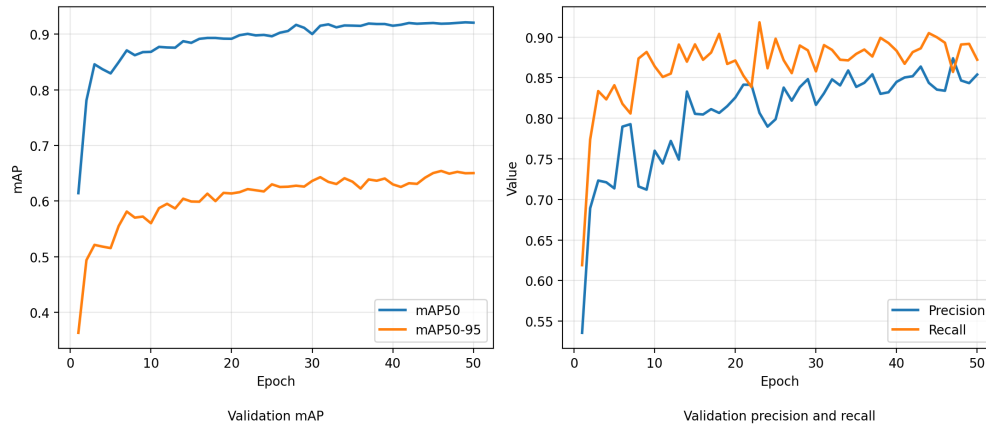


Figura 3: Evolución del mAP del detector sobre validación.

4.4 Resultados finales del detector

Cuadro 1: Resultados finales del detector sobre validación.

Métrica	Valor
Precision	0.85414
Recall	0.87606
mAP@50	0.91923
mAP@50-95	0.63881

En conjunto, estos resultados muestran que el detector alcanzó un desempeño sólido para el problema planteado. El valor alto de **mAP@50** sugiere que las detecciones son correctas en una proporción importante de los casos cuando se usa un criterio de solapamiento moderado, mientras que el valor de **mAP@50-95** refleja una caída esperable al exigir localizaciones más precisas en un rango más amplio de umbrales IoU. El balance entre precisión y recall también sugiere que el modelo logró una cobertura razonable sin volverse excesivamente conservador, aunque todavía persisten dificultades en clases visualmente más complejas o menos representadas, como *Platelets*.

4.5 Curvas de pérdida de entrenamiento y validación

La Figura 4 muestra la evolución de la pérdida de entrenamiento y validación de la ANN a lo largo de las épocas. Estas curvas permiten observar si el modelo converge de forma estable, si existe separación excesiva entre entrenamiento y validación, y si aparecen señales de sobreajuste durante el proceso de aprendizaje.

En las corridas realizadas, la ANN mostró una convergencia estable y sin oscilaciones severas. La pérdida de entrenamiento descendió progresivamente, mientras que la pérdida de validación se mantuvo en un rango bajo y relativamente estable. En la mejor configuración

observada, el modelo alcanzó un desempeño fuerte sobre validación, con `val_macro_f1` cercano a 0.99 y `val_accuracy` cercana a 0.998. Esto sugiere que la red fue capaz de aprender una representación discriminativa útil para el problema planteado.

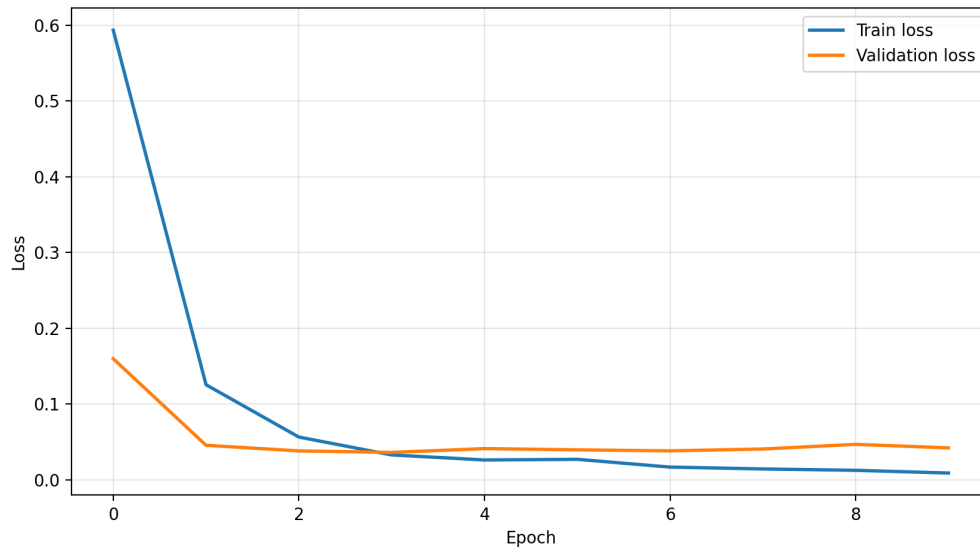


Figura 4: Training and validation loss curves for the ANN.

4.6 Análisis de entrenamiento

En términos generales, el entrenamiento de la ANN fue estable y mostró convergencia rápida. No se observaron señales fuertes de inestabilidad numérica ni divergencia de la función de pérdida. Asimismo, aunque el modelo alcanzó métricas muy altas, estas deben interpretarse con cautela, ya que la clasificación se realiza sobre recortes celulares ya extraídos y el dataset BCCD presenta correlación intra-imagen, lo cual tiende a producir una validación optimista.

La exploración de hiperparámetros mostró además que cambios suaves en `lr` y `dropout` no siempre producían diferencias grandes, mientras que cambios más marcados en capacidad y regularización sí modificaban el desempeño. En particular, una arquitectura intermedia con regularización fuerte resultó superior a configuraciones más débiles o menos controladas, lo cual sugiere que el modelo se benefició de una combinación equilibrada entre capacidad representacional y control de sobreajuste.

5. Evaluación comparativa de clasificadores

5.1 Metodología de comparación

Ambos clasificadores fueron comparados usando exactamente el mismo protocolo experimental, con el fin de que las diferencias observadas se debieran al modelo y no a cambios en los datos o en la representación de entrada. En todos los casos se utilizó el conjunto `train` para

entrenamiento y `valid` para validación, manteniendo fijo el extractor de características y la estrategia de preprocesamiento.

Elemento	Configuración
Dataset	BCCD
Split de entrenamiento	<code>data/bccd/train</code>
Split de validación	<code>data/bccd/valid</code>
Unidad de entrada	Recortes celulares obtenidos desde las anotaciones YOLO del dataset
Extractor de características	ResNet18 preentrenada, truncada en <code>layer3</code>
Dimensión del vector de features	256
Normalización de features	Habilitada
Estrategia de reducción dimensional	<code>none</code> (sin proyección en la configuración final comparada)
Clasificadores comparados	ANN y SVM
Métricas principales	<code>val_macro_f1</code> , <code>val_accuracy</code> , precisión por clase, recall por clase y matriz de confusión
Criterio principal de selección	<code>val_macro_f1</code>

Cuadro 2: Configuración utilizada para la comparación entre ANN y SVM.

Como nota metodológica, esta comparación se realizó sobre recortes celulares ya localizados a partir de las anotaciones del dataset, por lo que evalúa estrictamente el desempeño de clasificación y no el comportamiento end-to-end del pipeline completo.

5.2 Matriz de confusión de la ANN

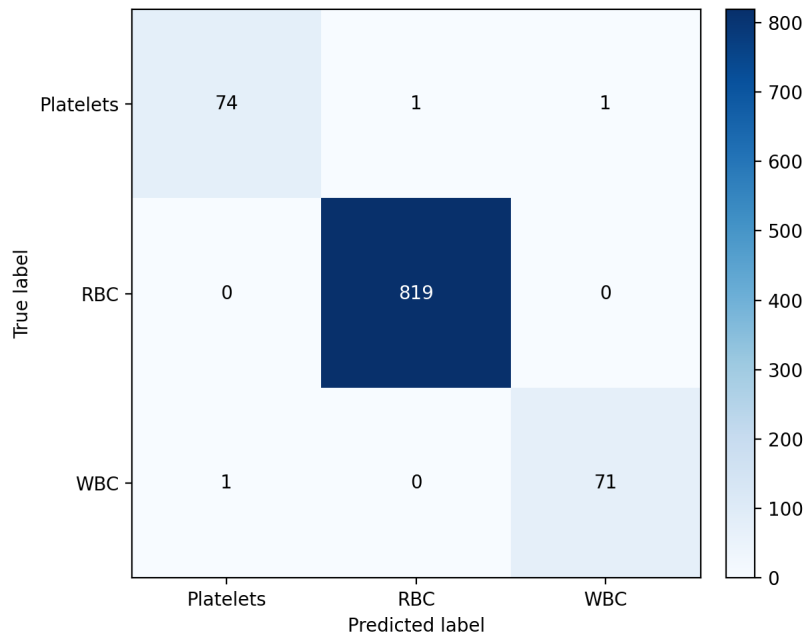


Figura 5: Matriz de confusión de la ANN.

5.3 Matriz de confusión de la SVM

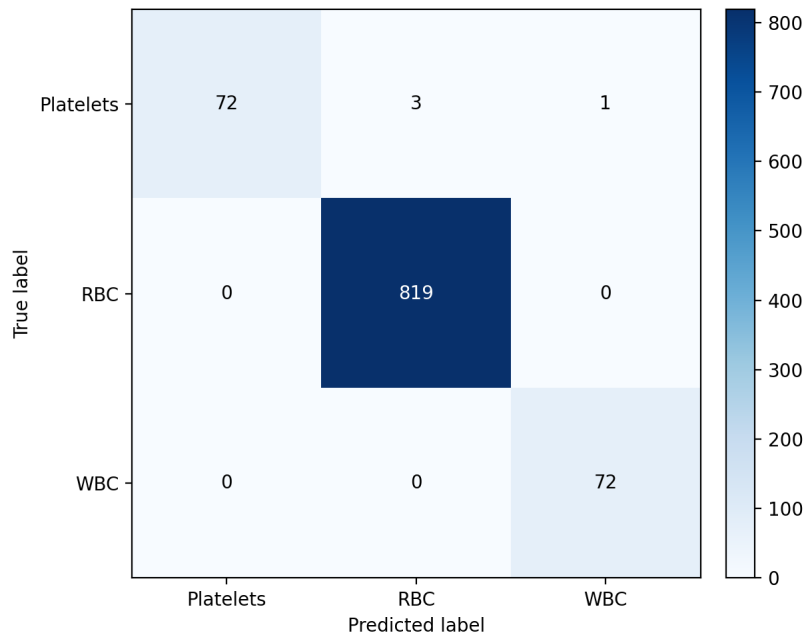


Figura 6: Matriz de confusión de la SVM

5.4 Métricas por clase

Clasificador	Clase	Precisión	Recall
ANN	WBC	0.9861	0.9861
ANN	RBC	0.9988	1.0000
ANN	Platelets	0.9867	0.9737
SVM	WBC	0.9863	1.0000
SVM	RBC	0.9964	1.0000
SVM	Platelets	1.0000	0.9474

Cuadro 3: Comparación de métricas por clase para ambos clasificadores

5.5 Comparación global

Clasificador	Accuracy	Macro F1
ANN	0.9969	0.9885
SVM	0.9959	0.9881

Cuadro 4: Comparación global entre ANN y SVM

La comparación global muestra que ambos clasificadores alcanzaron resultados muy competitivos, pero la ANN obtuvo el mejor desempeño general, tanto en **accuracy** como en **macro-F1**. Aunque la diferencia absoluta entre ambos modelos fue pequeña, la ANN presentó un comportamiento ligeramente más robusto y balanceado en clases minoritarias, por lo que se seleccionó como clasificador principal para la integración final del pipeline.

5.6 Análisis comparativo

En términos globales, la ANN obtuvo el mejor desempeño entre los dos clasificadores evaluados. Aunque ambas alternativas alcanzaron métricas altas sobre validación, la ANN superó ligeramente a la SVM tanto en **accuracy** como en **macro-F1**, lo que sugiere una mejor capacidad para mantener un desempeño balanceado entre las tres clases. Esta diferencia, aunque pequeña en valor absoluto, fue consistente con la selección final del modelo para la integración del pipeline.

En cuanto al comportamiento por clase, ambas aproximaciones mostraron resultados particularmente fuertes sobre *RBC*, la clase más abundante del dataset. Esto era esperable debido a que la distribución de datos favorece fuertemente esta categoría y a que sus patrones visuales son relativamente consistentes. Sin embargo, las diferencias más relevantes aparecieron en las clases minoritarias, especialmente *Platelets*. La ANN mostró un equilibrio más estable entre precisión y recall, mientras que la SVM alcanzó una precisión perfecta en *Platelets* pero con

un recall menor, lo que indica un comportamiento más conservador al asignar esa clase. En otras palabras, la SVM cometió menos falsos positivos en *Platelets*, pero dejó escapar más ejemplos reales de esa categoría.

Respecto a la SVM, la comparación entre kernels mostró que el kernel lineal fue suficiente para capturar la estructura del problema bajo la representación de características utilizada. En la exploración realizada, el kernel RBF no aportó una mejora clara frente al lineal, y en varias corridas incluso mostró un desempeño ligeramente inferior. Esto sugiere que, una vez extraídas las características profundas mediante ResNet18, las clases ya quedaban suficientemente separadas en un espacio donde una frontera aproximadamente lineal era adecuada. Desde este punto de vista, la representación aprendida por el extractor resultó más determinante que la complejidad geométrica adicional que pudiera aportar un kernel no lineal.

Finalmente, la separabilidad observada entre clases puede interpretarse a partir de dos factores principales. Primero, el extractor profundo generó descriptores suficientemente discriminativos para distinguir patrones celulares relevantes. Segundo, el problema de clasificación se resolvió sobre recortes ya localizados desde las anotaciones del dataset, lo que simplifica considerablemente la tarea con respecto al escenario end-to-end. Aun así, la persistencia de errores residuales en *Platelets* y, en menor medida, *WBC*, indica que las clases minoritarias siguen siendo las más sensibles a pequeñas diferencias de capacidad del modelo y al desbalance del conjunto de datos.

6. Justificación empírica de la mitigación de la alta dimensionalidad

6.1 Problema

Los clasificadores posteriores reciben como entrada vectores de características extraídos con ResNet18. Al truncar el extractor en `layer3`, cada recorte celular queda representado por un vector de 256 dimensiones. Dado que las clases minoritarias aportan menos muestras efectivas y que múltiples recortes provienen de un conjunto limitado de imágenes microscópicas, era necesario evaluar un mecanismo explícito para mitigar el riesgo de sobreajuste asociado a esta dimensionalidad.

6.2 Mecanismo evaluado

Para cumplir con FR-7 se implementó una proyección lineal opcional como mecanismo explícito de mitigación de dimensionalidad. Sea $x \in \mathbb{R}^{256}$ el vector de características producido por el extractor; la estrategia evaluada consistió en aprender una transformación lineal de la forma $z = Wx + b$, con $z \in \mathbb{R}^{64}$. En la implementación, este mecanismo se controló mediante el parámetro `projection_dim`, comparando configuraciones sin proyección (`projection_dim = null`) y con proyección (`projection_dim = 64`).

6.3 Evaluación mediante sweeps

La estrategia fue incorporada en sweeps de W&B para ambos clasificadores, ANN y SVM, manteniendo constante el resto del protocolo experimental: mismos splits de entrenamiento y validación, mismo extractor, misma normalización y mismo esquema de evaluación. De esta forma, el efecto de la proyección lineal pudo aislarse sin introducir factores de confusión adicionales.

Clasificador	Configuración	Feature dim	Accuracy	Macro F1
ANN	Sin proyección	256	0.99690	0.98854
ANN	Con proyección (<code>projection_dim = 64</code>)	64	0.99483	0.98555
SVM	Sin proyección	256	0.99586	0.98808
SVM	Con proyección (<code>projection_dim = 64</code>)	64	0.99483	0.98555

Cuadro 5: Resultados de los sweeps de FR-7 para comparar configuraciones con y sin proyección lineal.

6.4 Decisión final

Como se observa en la Tabla 5, la proyección lineal redujo efectivamente la dimensionalidad de entrada de 256 a 64, pero no mejoró el desempeño de validación en ninguno de los dos clasificadores. En ambos casos, las configuraciones sin proyección obtuvieron mejores valores de `accuracy` y `macro-F1`. Por ello, la proyección lineal no fue adoptada como configuración final del sistema.

7. Barrido de hiperparámetros con W&B (FR-11)

7.1 Metodología del barrido

Para cumplir con FR-11 se realizaron barridos sistemáticos de hiperparámetros mediante la funcionalidad de *sweeps* de Weights & Biases (W&B) para ambos clasificadores del sistema: una red neuronal artificial (ANN) y una máquina de soporte vectorial (SVM). En ambos casos, las configuraciones fueron evaluadas utilizando exclusivamente el conjunto de validación, manteniendo fijo el protocolo de extracción de características y registrando automáticamente cada corrida en la plataforma.

Los sweeps se ejecutaron sobre la misma representación de entrada para ambos clasificadores: recortes celulares obtenidos desde las anotaciones del dataset BCCD, procesados con un extractor `ResNet18` truncado en `layer3`, con normalización habilitada y sin proyección adicional en la comparación principal de FR-11. La métrica principal de selección fue `val_macro_f1`.

7.2 Resumen de sweeps ejecutados

Sweep	Clasificador	Método	Hiperparámetros explorados
fr11-ann-wide-bayes	ANN	Bayes	hidden_dims, dropout, lr, weight_decay, epochs,
fr11-svm-wide-bayes	SVM	Bayes	kernel, C, gamma

Cuadro 6: Resumen de los sweeps ejecutados para cumplir FR-11.

Clasificador	Run ID	Accuracy	Macro F1
ANN	ann_d06765ed	0.99897	0.99549
SVM	svm_fe245463	0.99586	0.98808

Cuadro 7: Mejores resultados obtenidos a partir de los sweeps de FR-11.

8. Resultados asociados a FR-14

8.1 Versión original del requisito

La formulación original de FR-14 hacía referencia a resultados de inferencia estadística sobre un conjunto sintético. Bajo esa versión, el énfasis estaba en estudiar alertas o desviaciones respecto a un baseline en imágenes artificiales.

8.2 Versión actualmente aplicada en el proyecto

En la versión finalmente aplicada, FR-14 se reinterpretó sobre datos reales del dataset BCCD. En lugar de evaluar un conjunto sintético, el análisis se centró en comparar tres niveles de etiquetado sobre imágenes reales:

- la etiqueta real anotada en el dataset (*ground truth*),
- la etiqueta predicha por el detector YOLO,
- la etiqueta predicha por el clasificador posterior sobre cada recorte detectado.

Este cambio permitió alinear la evaluación con el flujo real del sistema y con el criterio actualizado indicado por el profesor.

8.3 Metodología de evaluación usada

La evaluación de FR-14 se realizó sobre el conjunto de validación, utilizando un procedimiento de emparejamiento entre detecciones y anotaciones reales basado en IoU. Para que una

detección se considerara correctamente emparejada, debía existir solapamiento suficiente con una anotación real de la misma clase. A partir de este procedimiento se contabilizaron tres tipos de resultados:

- **Matched:** detecciones emparejadas correctamente con una anotación real,
- **FP:** detecciones sin correspondencia válida (*false positives*),
- **FN:** anotaciones reales no detectadas (*false negatives*).

Sobre estos conteos se calcularon **precision**, **recall** y **F1**. Además, se estudió el efecto del umbral de confianza del detector, ya que este parámetro afecta directamente el balance entre falsos positivos y falsos negativos. Para ello se evaluaron tres valores de `conf_threshold`: 0.25, 0.40 y 0.50.

8.4 Resultados

Umbral	Matched	FP	FN	Precision	Recall	F1
0.25	883	412	84	0.6819	0.9131	0.7807
0.40	797	239	170	0.7693	0.8242	0.7958
0.50	703	152	264	0.8222	0.7270	0.7717

Cuadro 8: Resultados principales asociados a FR-14 para distintos valores de `conf_threshold`.

Los resultados muestran un comportamiento consistente con el compromiso clásico entre precisión y recall. Con `conf_threshold = 0.25`, el sistema obtiene mayor recall, pero a costa de un número muy alto de falsos positivos. En cambio, con `conf_threshold = 0.50`, la precisión mejora, pero el número de falsos negativos crece de forma importante. El mejor equilibrio se obtuvo con `conf_threshold = 0.40`, que alcanzó el mayor valor de **F1** sobre validación.

En consecuencia, el umbral seleccionado para FR-14 fue `conf_threshold = 0.40`, ya que proporcionó el mejor balance entre cobertura de detecciones reales y control de detecciones espurias.

8.5 Discusión

El valor agregado del segundo nivel de clasificación no consiste únicamente en repetir la decisión de YOLO, sino en refinar la etiqueta final sobre cada recorte detectado mediante un clasificador entrenado específicamente sobre características profundas. Esto permite separar conceptualmente dos fuentes de error diferentes: los errores de detección del detector y los errores de clasificación del modelo posterior.

La evaluación de FR-14 mostró que una parte importante del desempeño end-to-end depende del punto de operación del detector, particularmente del umbral de confianza utilizado. Por ello, el segundo nivel de clasificación aporta valor cuando las detecciones son razonablemente correctas, pero no puede compensar completamente errores severos de localización o exceso de falsos positivos generados por YOLO. En este sentido, FR-14 permitió demostrar que el sistema completo debe evaluarse como una cadena integrada y no solo a través de métricas de clasificación sobre recortes ideales.

8.6 Metodología de medición

Se realizó el benchmark sobre las 36 imágenes del conjunto de prueba (`data/bccd/test/images`). Para cada etapa se reporta el tiempo promedio por imagen, junto con su variabilidad, tras una corrida de calentamiento inicial para excluir el costo de inicialización de modelos y artifacts. Las mediciones se ejecutaron únicamente en CPU, ya que no se detectó disponibilidad de GPU en el entorno de ejecución.

En la etapa de detección y en el pipeline completo se incluyó la lectura de la imagen desde disco, por formar parte del flujo real de inferencia. En cambio, para extracción de características y clasificación se excluyó la carga de disco, midiendo únicamente el procesamiento sobre datos ya disponibles en memoria.

8.7 Hardware utilizado

- Procesador: AMD Ryzen 7 7735HS with Radeon Graphics
- Memoria RAM: 14 GiB
- GPU: no aplica
- Sistema operativo: Ubuntu 24.04.4 LTS (Noble Numbat)

8.8 Tiempos por etapa

Cuadro 9: Tiempos de ejecución por etapa principal del pipeline.

Etapas	Tiempo promedio
Detección (YOLO)	43.43 ms
Extracción de características	94.80 ms
Clasificación ANN	0.24 ms
Clasificación SVM	1.80 ms
Pipeline completo	132.08 ms